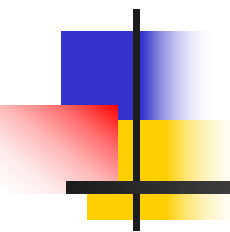


C语言与程序设计

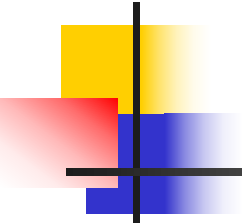
The C Programming Language



第7章 数组

华中科技大学计算机学院

卢萍



主要内容

- ◆ 一维/二维数组的声明、初始化和使用
- ◆ 数组作为函数参数的使用
- ◆ 字符串数组
- ◆ 多维数组

7.1 数组概述

用来描述一群有联系的同类型数据集合

```
#define SIZE 30
```

```
int score[SIZE];
```

score是含有30个元素的int型数组

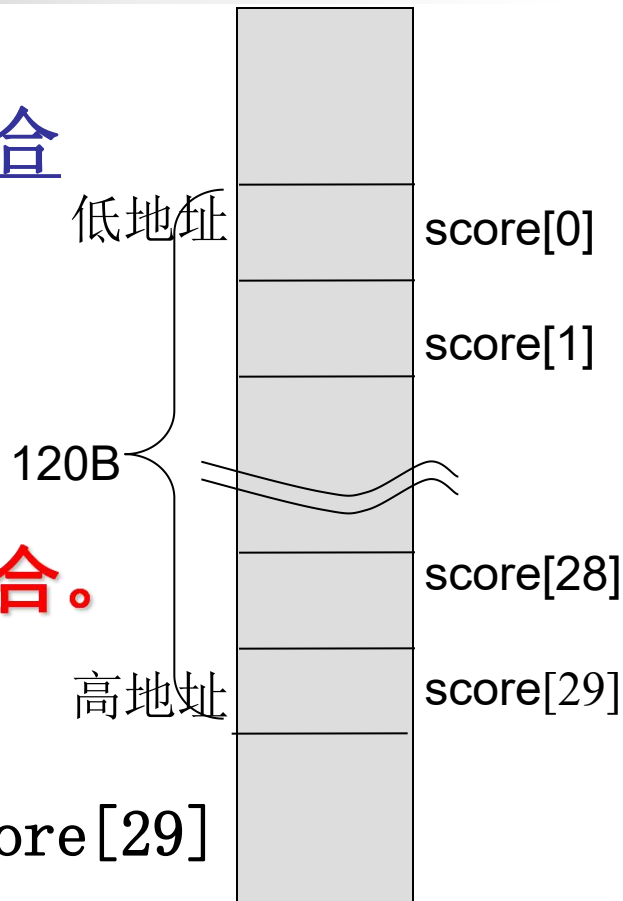
数组：是固定数量的同类型元素的集合。

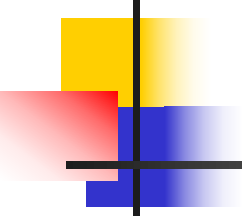
➤ 数组元素的下标从0开始

数组score的30个元素是：score[0]～score[29]

➤ 各个元素在内存中连续存放

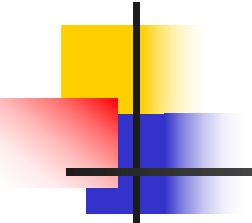
数组score所占内：sizeof (int) * SIZE





7.2 一维数组

- 只有一个下标
- 可用于表示一个线性的数据队列



一维数组的声明

存储类型 类型说明符 数组名[常量]={初值表} ;

- 蓝色部分可选
- 不能是 register
- `int score[SIZE];`
- `static int y[10];`
- `extern double s[2];`
- `int n=4, a[n];` **// C99支持**

一维数组元素的引用

数组名[下标表达式]



整型表达式

整型常量

整型变量

含有运算符的整型表达式

值为整型的函数调用

$a[2]$ 、 $a[i]$ 、 $a[i+j]$ 、 $a[\max(a,b)]$

代表了一个元素， 等价一个同类型的变量。

下标值不要超过数组的范围

```
int a[4],i;
```

```
for(i=0;i<=4;i++)
```

```
    a[i]=i+1;
```

$a[4]=5;$

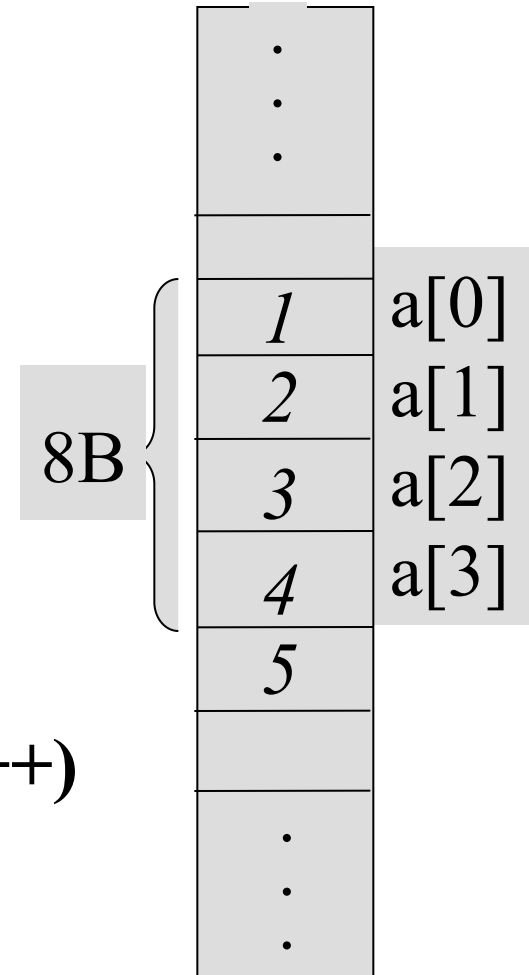
↑
下标越界

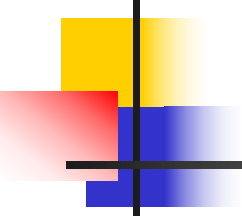
**破坏数组以外的其它变量的值,
可能造成严重后果**

```
int a[4],i;
```

```
for(i=0;i<4;i++)
```

```
    a[i]=i+1;
```





一维数组的初始化

存储类型 类型 数组名[常量表达式]={初值};

- 对全部元素赋初值（初值的个数与数组长度相同）

```
int x[5]={1,2,3,4,5};
```

```
<=> int x[ ]={1,2,3,4,5}; /*数组长度由初值个数确定*/
```

- 对部分元素赋初值(初值的个数小于数组长度)

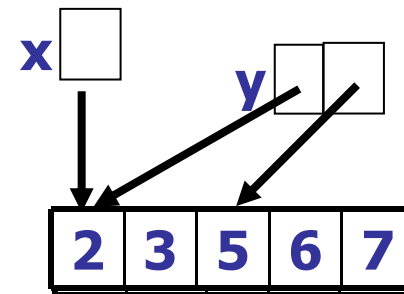
```
int z[6]={1,2,3,4}; /* 前4个下标变量赋值 */
```

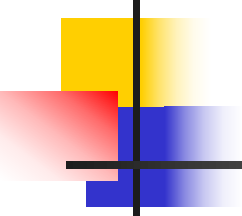
```
z[0]:1    z[1]:2    z[2]:3    z[3]:4
```

一维数组作为函数参数

```
void fun(int y[ ], int n)
{ int i;
  for(i=0; i<n; i++)  y[i]++;
}

int main(void)
{ int i,x[5]={1,2,3,4,5};
  fun(x,5);
  for(i=0;i<=4;i++) printf("%d",x[i]);
  fun(&x[2], 3);
  printf("\n");
  for(i=0;i<=4;i++) printf("%d",x[i]);
}
```

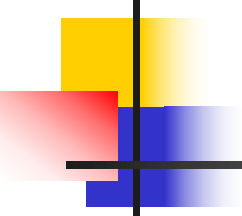




C 的参数传递：值传递

值 { 变量的值（传值）
变量的地址值（传址）

用数组名作参数时，则为地址传送，即实参数组的首地址传递给形参数组首地址。所以，实参、形参数组共享相同的内存单元。相当于是值的双向传递



排序问题

排序问题——计算机处理数据的一个重要问题

➤ 对 n 个整数排序，**将冒泡排序法定义成函数。**

- **函数名: *BubbleSort***

- **形式参数:**

带排序的若干整数: 数组 a

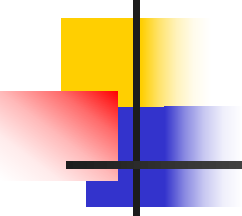
整数的个数: n

- **返回值: 无**

- **函数原型**

void BubbleSort(int a[], int n);

- **算法?**

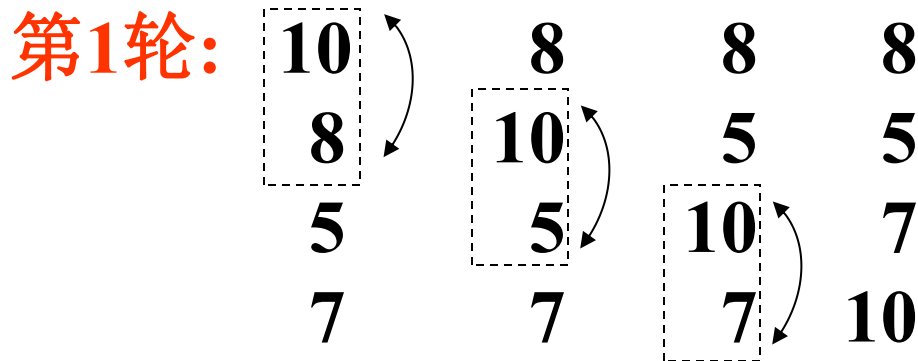


冒泡 (bubble) 排序法

基本思路：设有 n 个数，需将它们从小到大顺序排列。则：

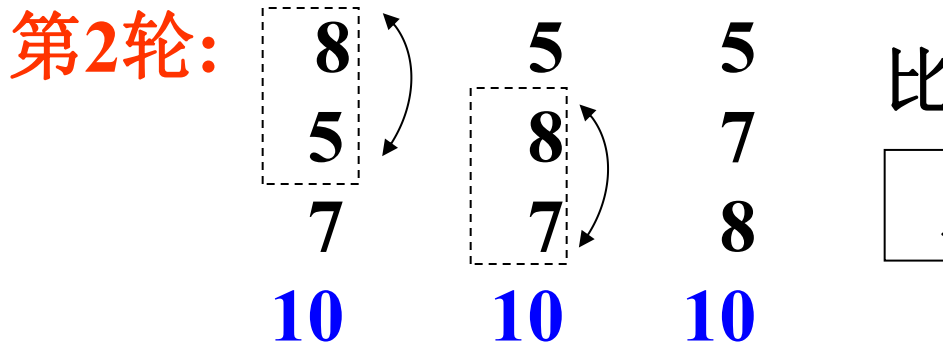
- 1. 对 n 个元素，依次比较相邻的两个数，小的调到前头，比较 $(n-1)$ 次后，最大的一个数“沉底”。
- 2. 对剩下的 $n - 1$ 个元素，比较 $(n-2)$ 次后，得到次大的数。
- 3. 对上述过程重复，直至剩下一个元素。

例: `int a[4] = {10 8 5 7};`

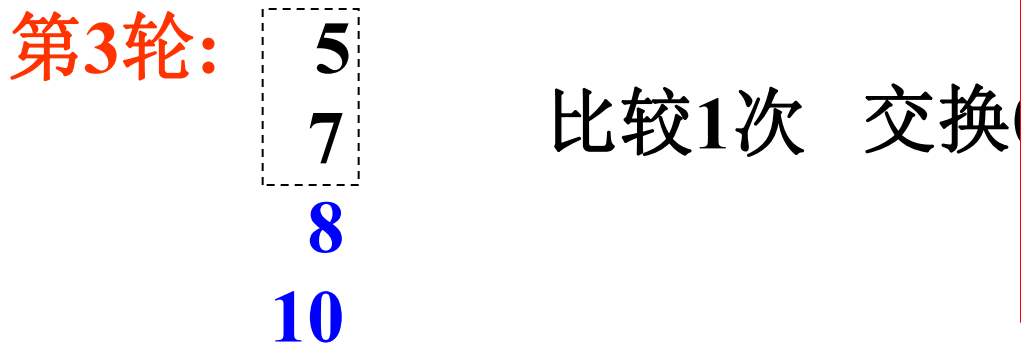


比较3次 交换3次

n 个数进行 ? 轮比较
`for(i=1; i<n; i++){`
 相邻元素的一一比较, 交换
 交换2次
`}`



第 i 轮中要进行 ? 次两两比较



`for(i=1; i<n; i++){`
 `for(j=0; j<n-i; j++){`
 `if(a[j]>a[j+1])`
 `a[j]与a[j+1]交换`
 }
`}`

```
#include<stdio.h>
```

```
#define N 10
```

```
void BubbleSort ( int a[ ],int n)
```

```
{
```

```
    int i, j, t;
```

```
    for (i=1; i<n; i++) /* 共进行n-1轮"冒泡" */
```

```
        for (j=0; j<n-i ; j++) /* 对两两相邻的元素进行比较 */
```

```
            if (a[j]>a[j+1] ) { t=a[j]; a[j]=a[j+1]; a[j+1]=t; }
```

```
    }
```

```
int main ( )
```

```
{ int x[N], i;
```

```
    printf (" please input %d numbers: \n ", N);
```

```
    for (i=0; i<N; i++) scanf(" %d ", &x[i]);
```

```
    BubbleSort (x, N) ;
```

```
    printf (" the sorted numbers: \n ");
```

```
    for (i=0; i<N; i++) printf("%d ", x[i]);
```

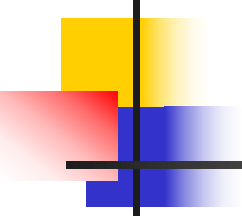
```
    return 0;
```

```
}
```

待排序元素
的个数

形参数组
不指定大小

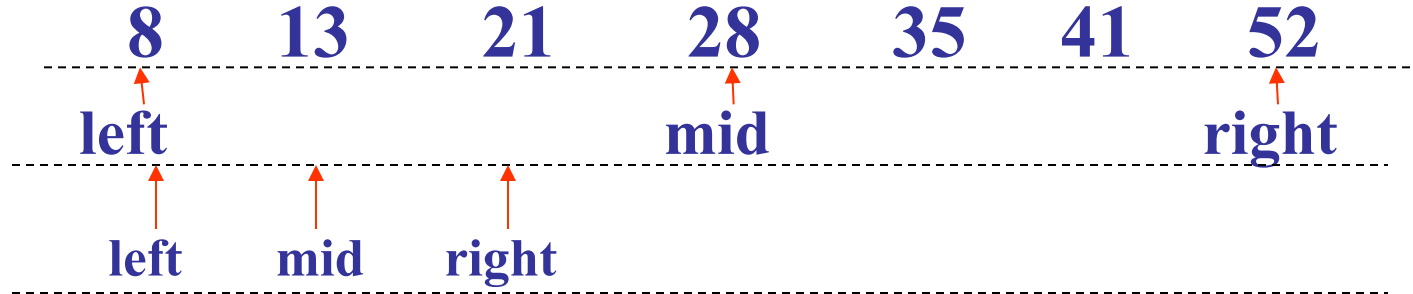
实参是数组名



基于分治策略的二分查找

- **二分查找（也称折半查找）** 算法的思路
 - 将已排好序的 n 个元素(存放于数组 a)分成两半，取 $a[n/2]$ 与 x 比较
 - 如果 $x=a[n/2]$ ，则找到 x ，算法结束
 - 如果 $x<a[n/2]$ ，则在数组 a 的前半部分继续查找 x
 - 如果 $x>a[n/2]$ ，则在数组 a 的后半部分继续查找 x
- **返回值**
 - 如果找到 x ，返回 x 在数组 a 中的位置（下标）
 - 如果没有找到，返回-1

在 $n=7$ 个数中 找 $x=14$



1) 令 $\text{left} = 0$, $\text{right} = n-1$

2) $\text{mid} = (\text{left} + \text{right}) / 2$

3) 将 x 与 $a[\text{mid}]$ 比较

◆ $x < a[\text{mid}]$, x 在 $a[0]$ 与 $a[\text{mid}-1]$ 范围内

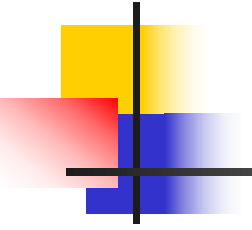
令 $\text{right} = \text{mid} - 1$

◆ $x > a[\text{mid}]$, x 在 $a[\text{mid}+1]$ 与 $a[\text{right}]$ 范围内

令 $\text{left} = \text{mid} + 1$

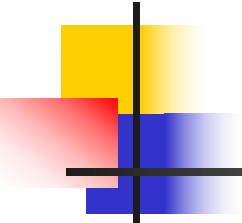
◆ $x == a[\text{mid}]$, x 在数组 a 中的下标为 mid

转2, 直至 $\text{left} > \text{right}$



二分查找函数

```
/* find x in a[0]~ a[n-1] ( sort ascending )  
return the subscript , or -1 if no match */  
  
int BinarySearch(int a[ ], int x, int n)  
{ int left=0, right=n-1, mid;  
  while(left<=right) {  
    mid=(left+right)/2; /* 计算中间元素的下标 */  
    if(x<a[mid])      right=mid-1; /* 在左半部找 */  
    else if(x>a[mid]) left=mid+1; /* 在右半部找 */  
    else return (mid); /* 找到, 返回下标 */  
  }  
  return -1; /* 没有找到, 返回-1 */  
}
```



二分查找函数

```
#include<stdio.h>
```

```
int main(void)
```

```
{
```

```
    int x[ ]={1,3,5,7,9,11,13,15,17,19}, index;
```

```
    index=BinarySearch(x,11,10);
```

```
    if(index!=-1)
```

```
        printf("find %d!\n",x[index]);
```

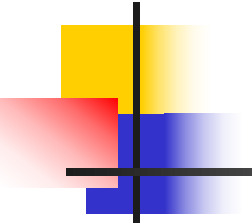
```
    else
```

```
        printf("not find!\n");
```

```
    return 0;
```

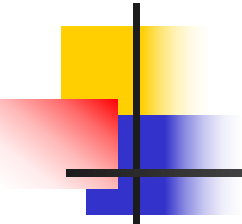
```
}
```

运行结果是：
find 11!



分治法

- 分治法的基本思想是，将一个大问题划分成若干子问题，这些子问题互相独立且与原问题相同。由分治法产生的子问题往往是原问题的较小模式，这就为使用递归技术提供了方便。在这种情况下，反复应用分治手段，可以使子问题与原问题类型一致而其规模却不断缩小，最终使子问题缩小到很容易直接求出其解。这自然导致递归过程的产生。
- 分治与递归像一对孪生兄弟，经常同时应用在算法设计之中，并由此产生许多高效算法。



分治法与递归

- **分治法**：将一个大问题划分成若干子问题，这些子问题互相独立且与原问题相同、规模缩小。
- 递归求解子问题。

用递归实现二分查找

8 13 21 28 35 41 52
↑ ↑ ↑
left mid right

在 $a[\text{left}] \sim a[\text{right}]$ 中找 x , 将 x 与 $a[\text{mid}]$ 比较

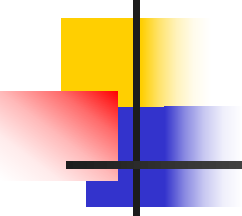
◆ $x < a[\text{mid}]$, 在 $a[\text{left}] \sim a[\text{mid}-1]$ 找

◆ $x > a[\text{mid}]$, 在 $a[\text{mid}+1] \sim a[\text{right}]$ 找

函数原型:

```
int BinarySearch(int a[ ], int x, int left, int right);
```

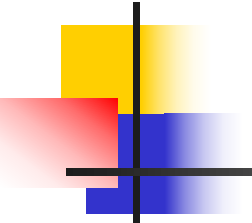
结束条件?



二分查找的递归形式

//在升序数组a[low]~a[high]中找x

```
int BinarySearch(int a[ ], int x, int left, int right)
{
    if(left>right) return -1;        // 没有找到
    int mid=(left+right)/2;
    if(x==a[mid]) return mid;        //找到x
    else if(x>a[mid]) //x在后半部分
        return BinarySearch(a, x, mid+1, right)
    else //x在前半部分
        return BinarySearch(a, x, front, mid-1);
}
```



分治法练习

1、使用分治的思想从一个无序的整数数组中查找最大值。

如果数组大小为1，则可以直接给出结果

如果数组大小为2，则一次比较即可结果

如果数组大小 >2 ，则分治(二分)，递归求解

```
int find_max(int a[],int low,int high);
```

2、基于分治策略的排序

参见教材5.4.5节

基于分治策略的排序（5.4.5节）

（1）分解：将数组分为两部分

给定数组 $a[\text{left}] \sim a[\text{right}]$ ，从中选择一个元素（称为分区元素），并把其余元素划分为两个子集合：

$a[\text{left}] \sim a[\text{split}-1]$ 、 $a[\text{split}]$ 、 $a[\text{split}+1] \sim a[\text{right}]$

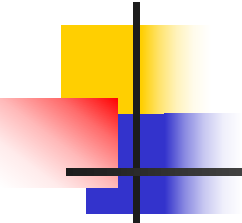
左边部分的所有元素都比右边部分的元素小

$v[] = \{ 6 \ 5 \ 2 \ 4 \ 7 \ 3 \ 1 \ 8 \}$

3 1 2 4 7 6 5 8

（2）递归求解

对两个子集合递归应用同一过程，当某个子集合中的元素个数小于2时，递归结束。



函数QuickSort

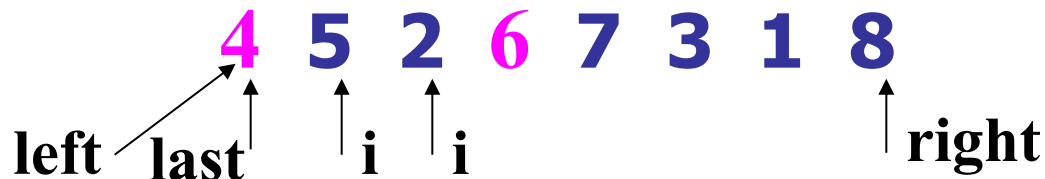
```
/* quick排序法 */  
void QuickSort(int a[ ],int left,int right)  
{  
    int split;    /* 分区位置 */  
    if(left<right) /* 待排序数组的元素个数至少为2 */  
    {  
        split=partition(a,left,right); /* 将数组元素分成两部分*/  
        QuickSort(a,left,split-1);    /* 对左边部分递归排序 */  
        QuickSort(a,split+1,right); /* 对右边部分递归排序 */  
    }  
}
```

分区 (方法1)

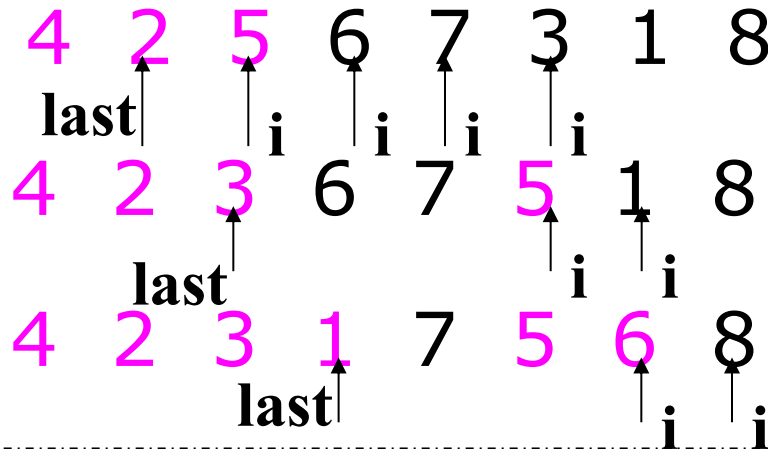
- 选择中间元素作为切分元素

$v[] = \{ 6 \ 5 \ 2 \ 4 \ 7 \ 3 \ 1 \ 8 \}$

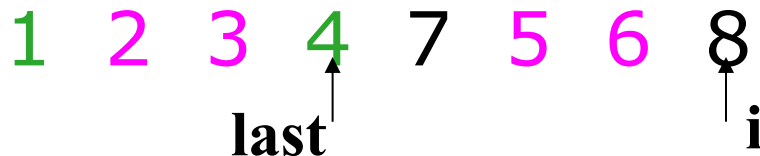
(1) 移切分元素到最左边位置

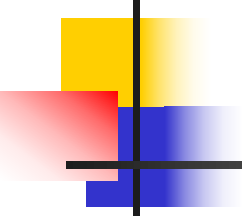


(2) 分区



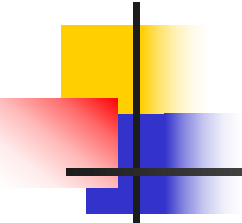
(3) 恢复切分元素到中间





函数partition_v1

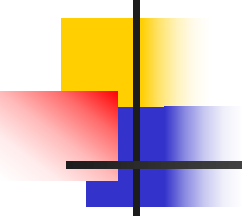
```
/* 将数组a中的元素a[left ]至a[right]分成左右两部分，  
   返回切分点的下标。 */  
int partition_v1(int a[ ],int left,int right )  
{ int i, last;    /*last为左边部分最后一个元素的位置*/  
  int split=(left+right)/2; /* 选择中间元素作为切分元素 */  
  swap(a, left, split); /* 移切分元素到最左边位置 */  
  last = left;    /* 初始化last */  
  for (i = left + 1; i <= right; i++) /* 分区：从左至右扫描 */  
    if (a[i] < a[left]) /* 小的数移到左边*/  
      swap(a, ++last, i);  
  swap(a, left, last); /* 将切分元素移到两部分之间*/  
  return last;  
}
```



函数swap

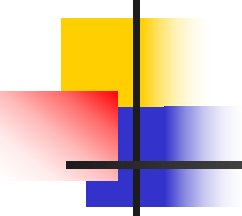
```
/* swap: 交换 v[i] and v[j] */  
void swap(int v[], int i, int j)  
{  
    int temp;  
    temp = v[i];  
    v[i] = v[j];  
    v[j] = temp;  
}
```

- `stdlib.h` 中有 函数 `qsort` 能对任意类型的对象排序。



函数partition_v2

```
int partition_v2(int a[ ],int left,int right )
{
    int i=left,j=right+1;
    int split=(left+right)/2; /* 选择中间元素作为切分元素 */
    swap(a,left,split); /* 将切分元素移到数组的开头 */
    for ( ; ; )
    {
        while(a[++i]<=a[left] && i <= right); /*从左至右扫描 */
        while(a[--j]> a[left]); /* 从右至左扫描 */
        if(i>=j) break; /* 扫描相遇（或交叉）结束循环 */
        swap(a,i,j); /* 交换左右两边的元素 */
    } /* j 是切分元素的位置 */
    swap(a,left,j); /* 将切分元素重新移到中间 */
    return j; /* 返回切分元素的下标 */
}
```



切分元素的选择

- 选择切分元素有很多种策略，最简单的方法是选用数组的第一个元素，该法对随机排列的数组很好，如果数据基本有序，则执行效率很差。上述程序中的方法可极大提高对有序或基本有序数组排序的效率。更加完善的策略是选择中间值，或至少是介于最大值和最小值之间的数值。
- 编写测试主函数用于输出排序结果。

7.3 二维数组

- 有两个下标
- 可用于表示数据阵列(二维数据表格)

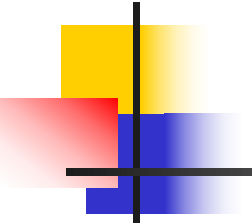
	语文	数学	英语	物理
01	85	91	80	78
02	82	95	88	92
...	...	...	...	...

如何描述上面的课程成绩表中的成绩数据？

用二维数组

```
int score[30][4];  /* score是30行4列的int型数组 */
```

二维数组还可以描述数学中的矩阵或行列式



二维数组的声明与使用

■ 说明

存储类 类型名 数组名[常量表达式][常量表达式]

■ 使用

数组名[行下标][列下标]

二维数组的存储结构

二维数组的存放方式：
按行存放

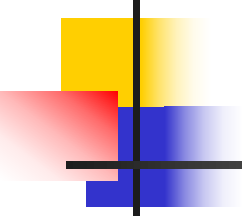
```
short x[2][3];
```

二维数组x的逻辑结构

x[0][0] =85	x[0][1] =91	x[0][2] =88
x[1][0] =82	x[1][1] =95	x[1][2] =88

二维数组x的物理存储结构

	•	
	•	
	•	
0xffd0	85	x[0][0]
0xffd2	91	x[0][1]
0xffd4	88	x[0][2]
0xffd6	82	x[1][0]
0xffd8	95	x[1][1]
0xffda	88	x[1][2]
	•	
	•	
	•	



二维数组的初始化

- **按照物理存储结构赋初值**（当一维数组处理）

```
int a[2][2]={85,91,82,95};
```

第1维大小的说明有时可以省略

```
int b[ ][3]={1,3,5,6,7};
```

- **按照逻辑结构**（按行） **赋初值**

可读性好

```
int x[2][3]={ { 85,91,0}, {82,95, 0}};
```

```
int b[ ][3]={ {1,3}, {5,6,7} };
```

举例

例：设全班同学修了高等数学、普通物理、c语言和英语4门课程并取得了成绩，要求计算每个同学的平均成绩，并输出成绩表。

序号	数学	物理	c	英语	平均成绩
01	91	78	91	80	78
02	95	92	95	88	92
	...	...	...	...	...

用二维数组表示上表。

```
int s[N][M+1]; /* N-人数， M-课程数*/
```

```

#include<stdio.h>
#define N 10      /* 人数 */
#define M 4       /* 课程数 */
double CalAverage (int [ ], int ) ; /* 计算平均值, 该函数定义略 */
int main(void)
{
    int x[N][M+1];
    int i,j;
    for(i=0;i<N;i++){
        for(j=0;j<M;j++){
            scanf("%d",&x[i][j]); /*输入成绩*/
            x[i][M]=CalAverage( x[i], M); /*计算学生的平均成绩*/
        }
        printf("\n");
        for(i=0;i<N;i++){ /*输出成绩表*/
            for(j=0;j<=M;j++){
                printf("%d\t",x[i][j]);
            }
            printf("\n");
        }
        return 0;
    }
}

```

$x[i] \Leftrightarrow \&x[i][0]$

// 扩展程序功能：加一行，储存每门课的平均成绩及总平均成绩

二维数组作为函数参数

```
void fun(int a[ ][M+1], int n)
{
    int i,j,sum;
    for(i=0;i<n;i++){
        for(sum=0,j=0;j<M;j++) sum+=a[i][j]; /* 学生i的成绩和 */
        a[i][j]=sum*1.0/j+0.5; /* 学生i的平均成绩(4舍5入) */
    }
}

int main(void)
{
    int x[N][M+1],i,j;
    for(i=0;i<N;i++)
        for(j=0;j<M;j++)
            scanf("%d",&x[i][j]); /*输入成绩*/
    fun(x,N);
}
```

待处理数组第1维的大小（即行数）

形参数组第1维不指定大小

实参是数组名

二维数组作为函数参数

```
void fun(int n, int m, int a[ ][m])
{
    int i,j,sum;
    for(i=0;i<n;i++){
        for(sum=0,j=0;j<m-1;j++) sum+=a[i][j]; /* 学生i的成绩和 */
        a[i][j]=sum*1.0/j+0.5; /* 学生i的平均成绩(4舍5入) */
    }
}
```

n为行数，m为列数

动态数组：第1维不指定大小，第2维是m

```
int main(void)
{
    int x[N][M+1],i,j;
    for(i=0;i<N;i++)
        for(j=0;j<M;j++)
            scanf("%d",&x[i][j]); /*输入成绩*/
    fun(N,M+1,x);
}
```

实参是数组名

构造n阶旋转方阵

1	2	3	4	5
16	17	18	19	6
15	24	25	20	7
14	23	22	21	8
13	12	11	10	9

(a) 顺转方阵

1	16	15	14	13
2	17	24	23	12
3	18	25	22	11
4	19	20	21	10
5	6	7	8	9

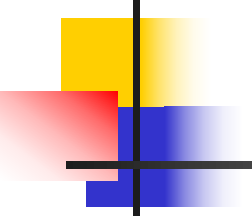
(b) 逆转方阵

7.4 n维数组

- 三维数组可以描述空间中的点集
- n维数组来描述n维线性空间中的n维向量

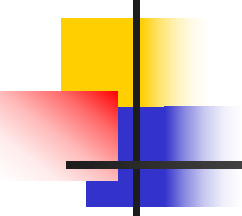
```
int a[ ][3][3]={ { {81,82,90},{73,94,90},{65,70,80} },  
                  { {80,86,87},{78,90,80},{89,60,70} }  
                };
```

第1页			80	86	87
			78	90	80
			89	60	70
第0页	81	82	90		
	73	74	90		
	65	70	80		



7.5 字符数组和字符串

- 类型为char的数组
- 存放字符数据
- 字符串——用一维字符数组存放
- 字符串数组——用二维字符数组存放
- 在末尾加一个空字符 ‘\0’ 来构造字符串
- 字符串常量 “……” 隐含有空字符 ‘\0’
- %s 也会自动加 ‘\0’
- 字符串的长度 = 字符串的存储长度-1

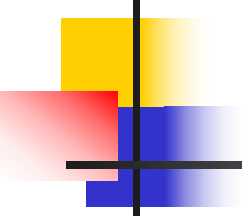


字符数组的声明和使用

```
char s[81];
```

***s是字符型数组，可以放入81个字符
可以表示最多由80个字符组成的字符串***

字符数组的最小长度=该字符串的长度+1



构造字符串

// 编程产生**26**个大写英文字母组成的字符串

```
#include <stdio.h>
int main(void)
{
    char Capital[27];
    int i ;
    for(i=0; i<26; i++)    Capital[i]='A'+i;
    Capital[26]='\0'; /*在末尾加    '\0'    来构造字符串*/
    puts(Capital);
    return 0;
}
```

字符数组的初始化

- 通过初始化列表(直接给出字符串中的各字符)

```
char s1[8]={ 'W', 'u', 'h', 'a', 'n', '\0' };
```

- '\0' 必须在初始化列表中显示给出
- 当数组长度未给出并且无'\0'时 ?

- 指定一个字符串常量 (常用)

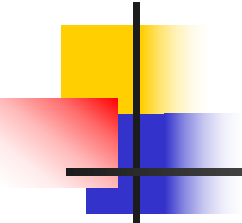
```
char s2[28]= "Computer Science" ;
```

- 末尾将自动加上一个 '\0'

当数组长度未给出时?

```
char s3[] = "Computer" ;
```

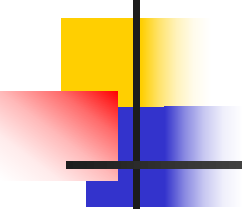
- 字符数组的长度等于字符串的存储长度



字符数组的初始化

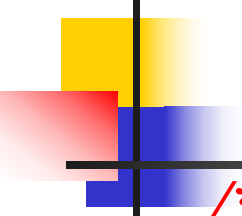
```
char s[] = "Com\0puter" ;
```

- `strlen(s)` = ? 3
- `sizeof s` = ? 10



字符串处理函数

- 串操作函数（<string.h>）
 - 求字符串长度(strlen) ---- 不计 ‘\0’
 - 字符串的拷贝(strcpy)
 - 字符串的比较(strcmp)
 - 字符串的连接(strcat)
 - 求字符串的子串(strstr)
 - 在字符串查找指定字符(strchr)
 - 将字符串反转等函数(strrev) ----非标准



求字符串长度的函数

/ gets length of s */*

```
int mystrlen(char s[ ])
```

```
{   int j=0;
```

```
    while(s[j] != '\0' )    j++;
```

```
    return j;
```

```
}
```

```
int main(void)
```

```
{ char str[100];
```

```
    gets(str); /* scanf("%s",str); */
```

```
    printf("length of the string is %d\n",mystrlen(str));
```

```
    return 0;
```

```
}
```

将字符串反转的函数

交换t[i]与t[j]

t:

c	h	i	n	a	\0	
---	---	---	---	---	----	--

i

j

t:

a	h	i	n	c	\0	
---	---	---	---	---	----	--

i

j

t:

a	n	i	h	c	\0	
---	---	---	---	---	----	--

i
j

i >= j 结束

```
void mystrev(char s[])
{
    int i, j; /*前、后指示器*/
    char c;
    for(i=0, j=strlen(s)-1; i<j; i++, j--)
    {
        c=s[i];
        s[i]=s[j];
        s[j]=c;
    }
}
```

字符串拷贝的函数

/ copies string s to t */*

```
void mystrcpy(char t[ ], char s[ ])
```

```
{
```

```
    int j=0;
```

```
    while( (t[j]=s[j]) !='\0' ) /* 可为 !=0 , 或去掉 !='\0' */
```

```
        j++;
```

```
}
```

```
int main(void)
```

```
{    char str1[30], str2[]= "there is a boat on the lake.";
```

```
    mystrcpy(str1,str2);
```

```
    puts(str1);
```

```
    return 0;
```

```
}
```

运行结果:

there is a boat on the lake.

字符串拷贝的函数

/ copies string s to t */*

void mystrcpy(char t[], char s[]) 对吗?

{

 int j=0;

 while((s[j] !='\0')

 t[j]=s[j++];

}

int main(void)

{ char str1[30]="there", str2[]= "is";

mystrcpy(str1,str2);

 puts(str1);

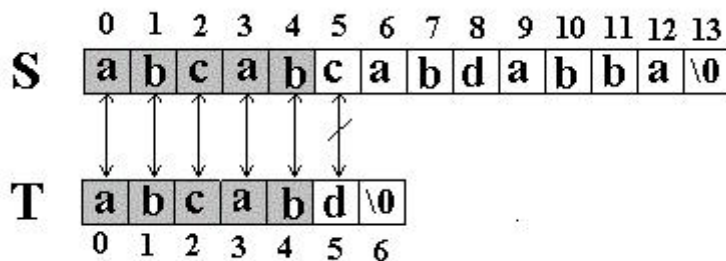
 return 0;

}

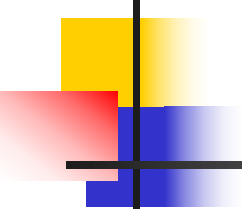
两个字符串比较函数

比较规则

从两个字符串的第一个字符起开始，按照字符**ASCII**码值的大小进行比较。



```
/* compares t to s  
return a value: =0 if s==t  
>0 if s>t  
<0 if s<t */  
  
int mystrcmp(char s[],char t[])  
{  
    int j=0;  
    while(s[j]==t[j] && s[j]!='\0')  
        j++;  
    return s[j]-t[j];  
}
```



例 验证密码

```
int main(void)
{
    char pw[]="1234",s[20];
    int count=3;
    do {
        printf("Input password\n") ;
        scanf("%s",s);
        count--;
    } while(mystrcmp(pw,s)&&count));
    if(mystrcmp(pw,s)
        return 1;
        .... // 进入系统
    return 0;
}
```

字符串的连接函数

/ appends s to t */*

```
char * mystreat(char t[],char s[])
```

```
{   int j=0,k=0;
```

```
    while(t[j++]!='\0') ;
```

```
    j--;
```

```
    while((t[j++]=s[k++])) ;
```

```
    return t;
```

```
}
```

```
void main(void)
```

```
{   char s1[80]="I like ", s2[]="the C programming. " ;
```

```
    mystreat(s1,s2);
```

```
    printf("%s\n",s1);
```

```
}
```

运行结果：

I like the C programming.

字符串的连接函数

/ appends s to t */*

```
char * mystreat(char t[],char s[])
```

```
{   int j=0,k=0;
```

```
    while(t[j++]!='\0') ;
```

```
    /*j--;*/           怎样?
```

```
    while((t[j++]=s[k++])) ;
```

```
    return t;
```

```
}
```

```
void main(void)
```

```
{   char s1[80]="I like ", s2[]="the C programming. " ;
```

```
    mystreat(s1,s2);
```

```
    printf("%s\n",s1);
```

```
}
```

运行结果:

I like

字符串的连接函数

/ appends s to t */*

```
char * mystreat(char t[],char s[])
```

```
{   int j=0,k=0;
```

```
    while(t[j++]!='\0') ;
```

```
    j--;
```

```
    while((t[j++]=s[k++])) ;
```

```
    return t;
```

```
}
```

```
void main(void)
```

```
{   char s1[ ]="I like ", s2[]="the C programming. " ;
```

```
    mystreat(s1,s2);
```

```
    printf("%s\n",s1);
```

```
}
```

缺省长度？

输出异常

删除字符串首尾空白字符的函数

" red pen " ----> "red pen"

1. " r red pen " 循环次数=新串字符数
= 原串字符数-首部空白字
2. " r ered pen " 符数-尾部空白字符数

3. " reded pen " 算法步骤:

.....
(1) 计算首部空白字符数
(2) 计算尾部空白字符数

7. " red penen " (3) 计算新串字符数

(4) 循环

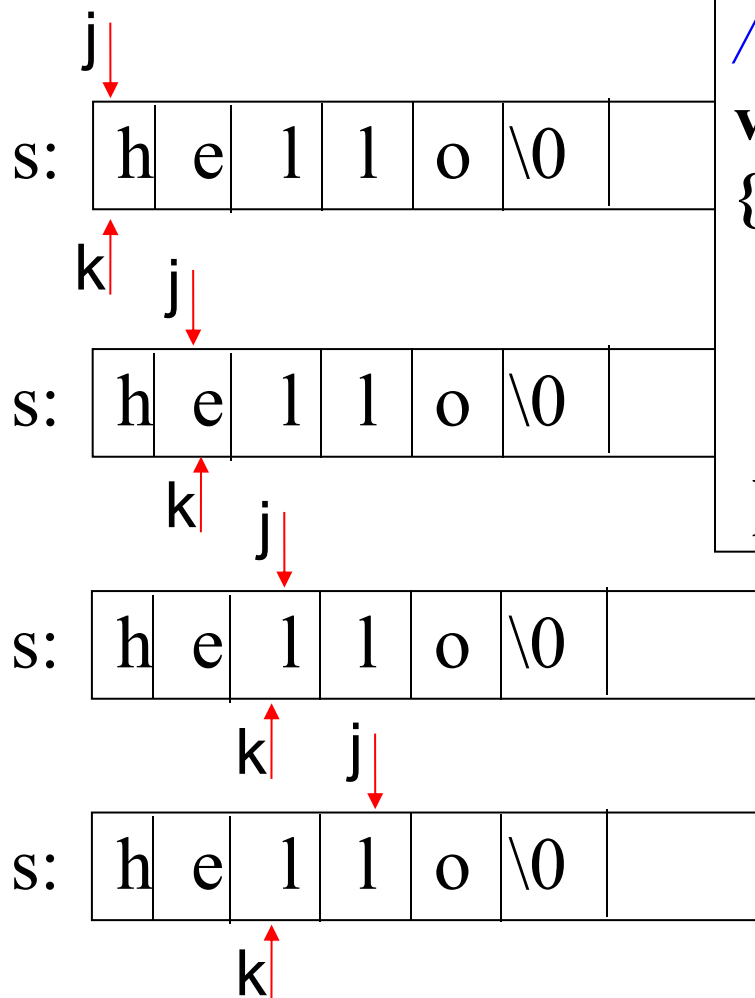
3. " red pen\0 " (5) 加 '\0'

删除字符串首尾空白字符的函数

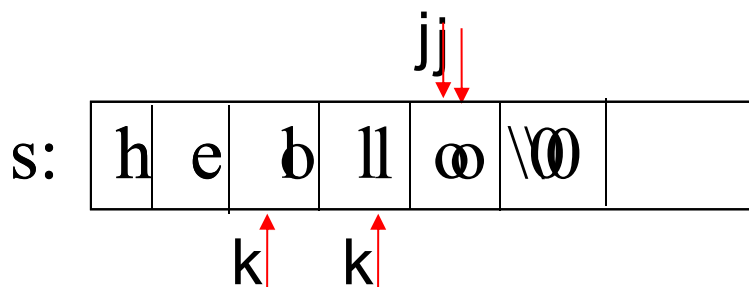
/* deletes spacing of beginning and end in s */

```
int trim(char s[])
{   int i, num, j=0, k=0, L=strlen(s);
    while(s[j]==' ' || s[j]=='\t' || s[j]=='\n' || s[j]=='\r')
        j++;           /* j计算首部空白字符的个数 */
    i=L-1;             /* i为字符串最后一个字符('\0'前面)的下标 */
    while(s[i-k]==' ' || s[i-k]=='\t' || s[i-k]=='\n' || s[i-k]=='\r')
        k++;           /* k计算尾部空白字符的个数 */
    num=L-j-k;
    for(i=0;i<num;i++)
        s[i]=s[i+j];
    s[num]='\0'; /*在末尾加 '\0' 来构造新字符串*/
    return strlen(s);
}
```

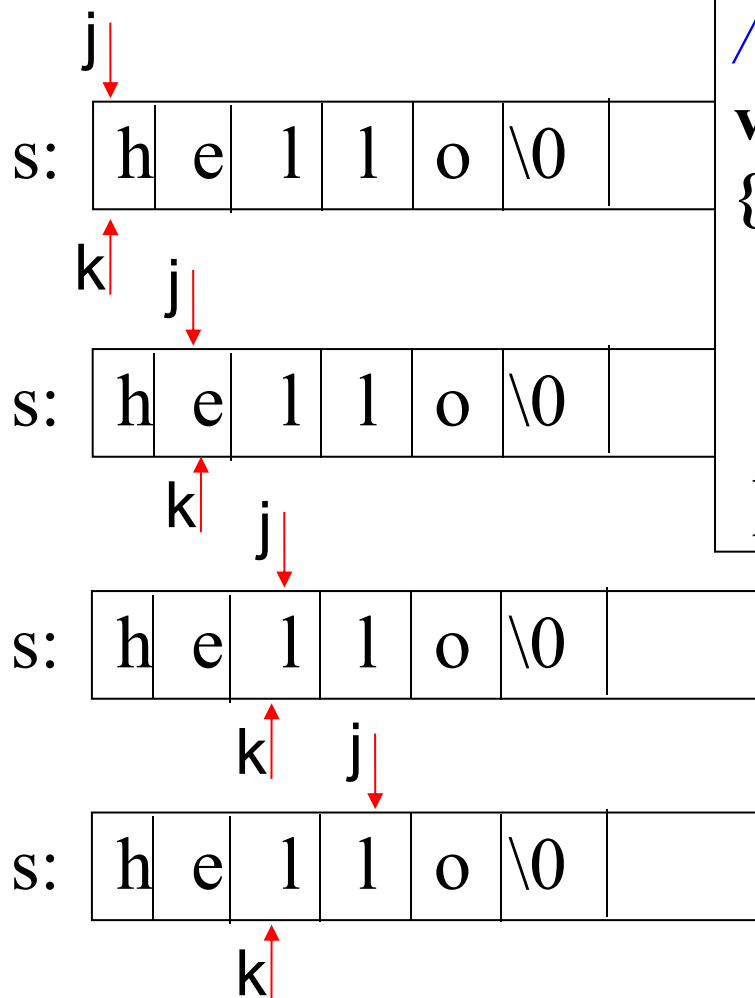
从串s中删除所有与给定字符c相同的字符



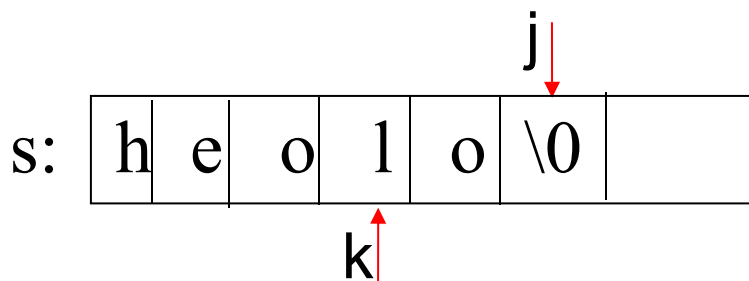
```
/* deletes all character c in s */
void strdelc(char s[ ],char c)
{ int j, k ;
  for(j=k=0;s[j]!='\0' ;j++)
    if(s[j]!=c) s[k++]=s[j];
  s[k]='\0' ;
}
```



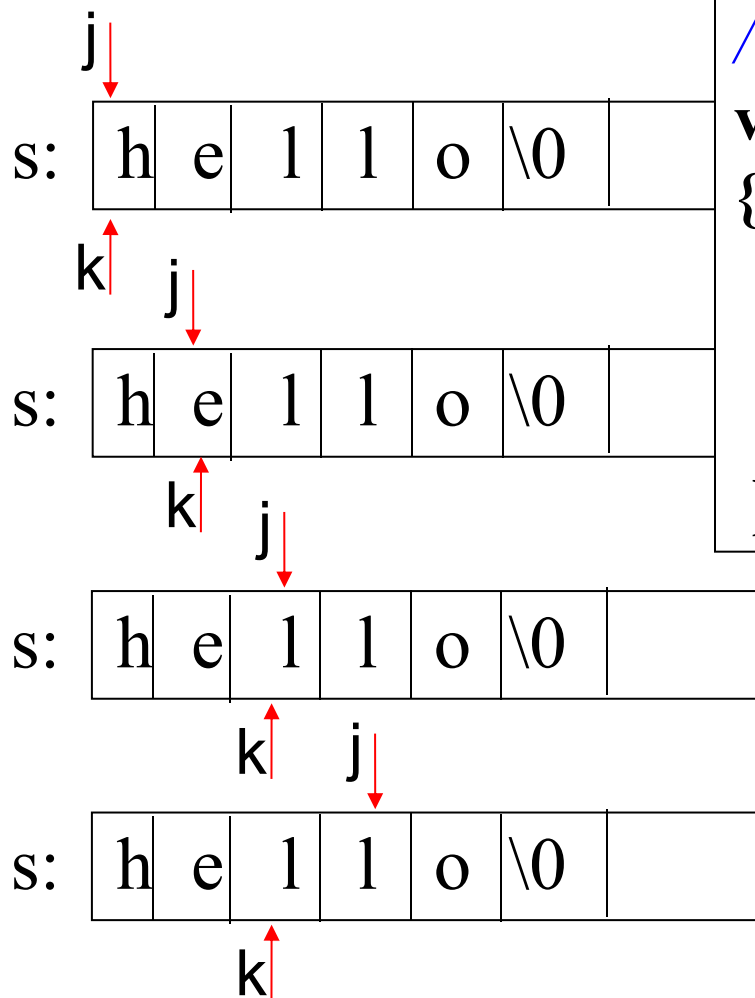
从串s中删除所有与给定字符c相同的字符



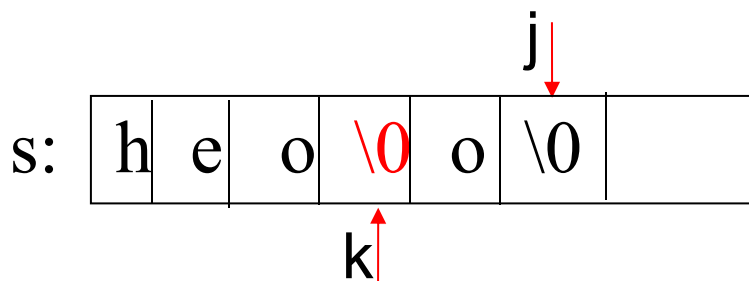
```
/* deletes all character c in s */  
void strdelc(char s[ ],char c)  
{ int j, k ;  
  for(j=k=0;s[j]!='\0' ;j++)  
    if(s[j]!=c) s[k++]=s[j];  
  s[k]='\0' ;  
}
```

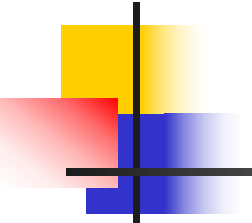


从串s中删除所有与给定字符c相同的字符



```
/* deletes all character c in s */  
void strdelc(char s[],char c)  
{ int j, k ;  
  for(j=k=0;s[j]!='\0' ;j++)  
    if(s[j]!=c) s[k++]=s[j];  
  s[k]='\0' ;  
}
```





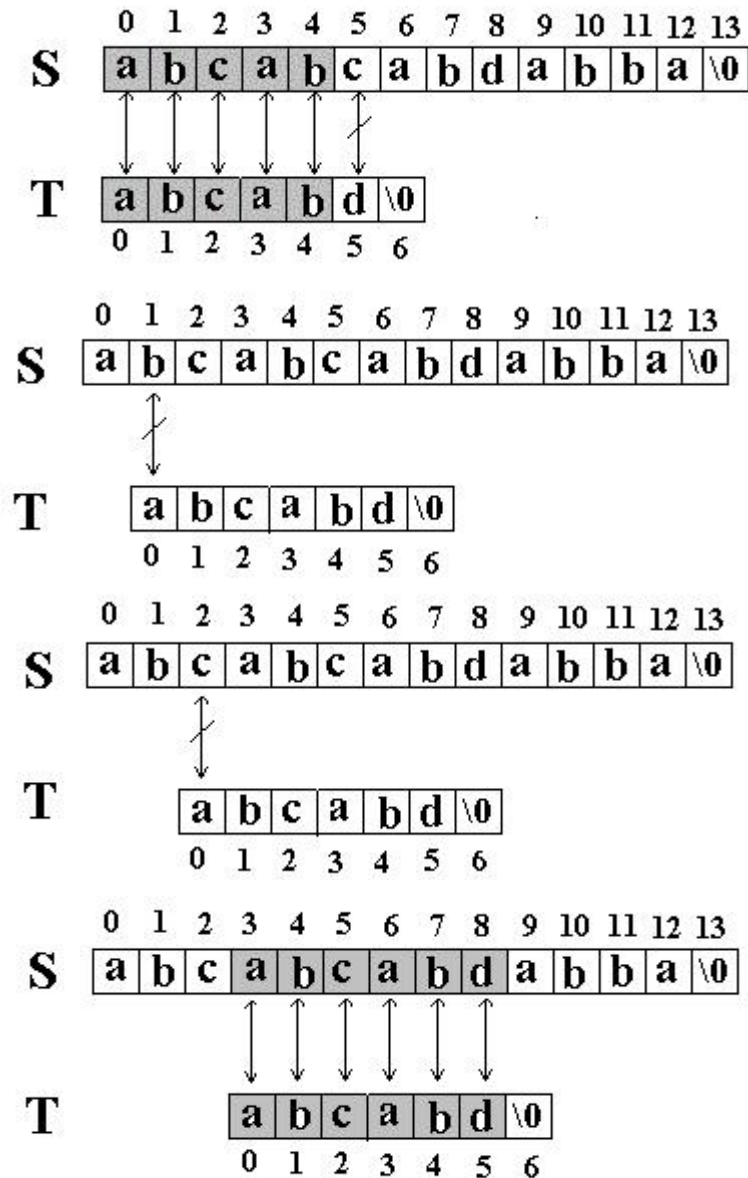
字符串匹配

字符串匹配——文本处理的一个重要领域

指从文本中找出给定字符串(称为模式)的一个或所有出现的位置----**求字符串s(文本)子串t(模式)**

❖ **strstr函数功能:** 如果字符串t是字符串s的子串, 则返回t在s中第一次出现时的位置(即下标)。 否则, 返回-1。

暴力法找子串



```
for (i = 0; s[i] != '\0'; i++) {  
    // 开始第 i 轮匹配  
    for (j=i, k=0; t[k]!='\0' ; j++, k++)  
        if(s[j]!=t[k])  
            break;    //结束本轮匹配  
    // 判断本轮匹配是否成功  
    if (t[k] == '\0')  
        return i;    //匹配成功 返回下标  
}  
return -1;    // 不存在子串, 返回 -1
```

BF(Brute Force) 暴力求解法、穷举

KMP(Knuth-Morris-Pratt)

BM(Boyer-Moore):文本编辑器常用

找子串函数

/*sstr: return index of t in s, -1 if none */

int mystrstr(char s[], char t[])

{

int i, j, k;

for (i = 0; s[i] != '\0'; i++) {

for (j=i, k=0; t[k]!='\0' ; j++, k++)

if(s[j]!=t[k])

break; /*结束本轮匹配*/

if (t[k] == '\0')

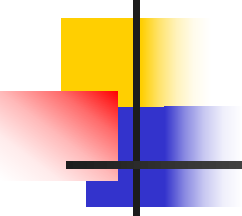
return i; /*匹配成功 返回下标*/

}

return -1;

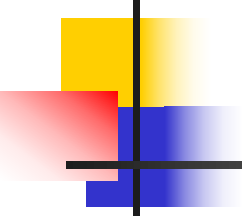
}

**for (j=i, k=0; t[k]!='\0' && s[j]==t[k]; j++, k++)
;**



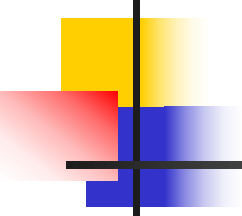
调用子串函数

```
int main(void)
{
    char s1[ ]="I like the C programming.", s2[]="ram";
    int i;
    i = mystrstr(s1,s2);
    if ( i>=0)
        printf("the sub-string's position is %d\n",i);
    else
        printf("\"%s\" does not occur in \"%s\"\n", s2,s1);
    return 0;
}
```



数字串与数之间转换的函数

- 串与数值之间的转换函数（<stdlib.h>）
 - 十进制串转换为整数(atoi)
 - 十进制串转换为长整数(atol)
 - 十进制串转换为浮点型数(atof)
 - 十六进制串转换为整数(atoi)
 - 整数转换为十进制串(itoa)



十进制串转换为整数 (atoi)

将一个十进制数字串转换成为对应的整数的函数

- **atoi函数功能**

- 将s字符数组中存放的一个十进制数字串转换成为对应的整数，并返回该整数

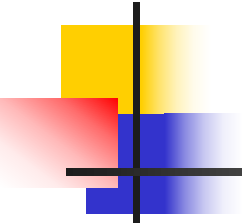
- **算法：**

- **ASCII码字符s[j]转换为对应数字**

- **$s[j] - '0'$**

- **本位乘以10加下一位的算法**

- **$54321 = (((((5) * 10 + 4) * 10 + 3) * 10 + 2 * 10) + 1$**



atoi函数

```
#define BASE 10
```

```
/* converts a string s to an int */
```

```
int myatoi(char s[])
```

```
{
```

```
    int j=0,num=0;
```

```
    for(;s[j]!='\0';j++)
```

```
        num=num*BASE +s[j]-'0';
```

```
    return num;
```

```
}
```

整数转换为十进制数字串 (itoa)

定义将一个十进制整数转换成为对应数字串的函数。**转换之后的数字串怎么返回？函数不能返回数组类型。**

➤ **形式参数：**

待转换的整数： n

转换结果串： 字符数组 s

➤ **返回值： 无**

➤ **函数原型**

`void myitoa(int  $n$ , char  $s$ []);`

➤ **算法： 分解出 n 的每一位数字存于数组 s 中。**

```
#define BASE 10
```

```
/* converts a integer n to a string s */
```

```
void myitoa(int n,char s[])
```

```
{   int sign,j=0;
```

```
    if((sign=n)<0) n=-n;
```

```
    while(n>0){
```

```
        s[j++] = n%BASE + '0';
```

```
        n/=BASE;
```

```
    }
```

```
    if(sign<0)
```

```
        s[j++] = '-';
```

```
    s[j] = '\0';
```

```
    strrev(s);
```

```
}
```

十六进制数字串转换为对应的整数 (htoi)

- 将一个以**0x** (**0X**) 为前缀的十六进制数字串转换成为对应的整数的函数
- **问题**
 - 当基数**BASE**大于**10**，如：**16**，由于十六进制数的表示形式，如'**a**'和'**A**'，'**b**'和'**B**'
 - 以及**0**到**f**或**F**在**ASCII**码表的编码不连续性
 - 因此需要在转换中进行一定的调整
- **htoi函数**
 - 将一个存放在字符数组**s**中的十六进制数字串转换成为对应的整数
 - 并且返回转换后的整数

```

int htoi(char s[])
{ int j=0,num=0;
  if(s[j]=='0' && (s[j+1]=='x' || s[j+1]=='X')) /*跳过前缀*/
    j+=2;
  else return -1; /* 不是十六进制数字串 */
  for(;s[j]!='\0';j++){
    if(s[j]>='0' && s[j]<='9') num=num*16+s[j]-'0';
    if(s[j]>='a' && s[j]<='f') num=num*16+s[j]-'a'+10;
    if(s[j]>='A' && s[j]<='F') num=num*16+s[j]-'A'+10;
  }
  return num;
}
  /* 不能处理前导空白符和符号位，如何改进？
   " 0XBE" , " -0XBE" */

```

二维字符数组

使用二维字符数组可以产生字符串数组

```
char text[25][80];
```

数组**text**可以存放**25**个字符串，每个字符串的最大长度为**79**。

■ 初始化

■ 与其它二维数组类似

```
char s[2][4]={ 'a','b','c','\0','d','e','f','\0'};
```

```
char s[2][4]={{'a','b','c','\0'},{'d','e','f','\0'}};
```

■ 用字符串对二维数组进行初始化（常用）

```
char devices[3][12]={ "hard disk" ," CRT" ," keyboard" };
```

```
<=> char devices[ ][12]={ "hard disk" ," CRT" ," keyboard" };
```

二维字符数组的使用

```
char devices[ ][10]={"hard disk", "CRT", "keyboard"};
```

h	a	r	d		d	i	s	k	\0
C	R	T	\0						
k	e	y	b	o	a	r	d	\0	

- 可以引用单个字符元素

`devices[2][3]` 值为 `'b'`

- 可以引用字符串

`devices[i]` 表示 `devices` 数组中第 `i` 行字符串的首地址

```
printf( "%s" , devices[1]); /* 输出 CRT */
```

字符串数组的输入输出操作

```
#include <stdio.h>
int main(void)
{
    int i;
    char devices[ ][10]={"hard disk", "CRT", "keyboard"};
    devices[0][0]='H'; /* "hard disk"变为"Hard disk"*/
    devices[2][0]='K'; /* "keyboard "变为"Keyboard "*/
    for(i=0;i<3;i++)
        printf("%s\n", &devices[i][0]); /* 同devices[i] */
    scanf("%s",devices[1]); /* 同 &devices[i][0] */
    for(i=0;i<3;i++)
        printf("%s\n", devices[i]);
    return 0;
}
```